

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
ProgramName: B. Tech		Assignment Type: Lab	AcademicYear: 2025-2026
CourseCoordinatorName		Venkataramana Veeramsetty	
Instructor(s)Name		Dr. V. Venkataramana (Co-ordinator)	
		Dr. T. Sampath Kumar	
		Dr. Pramoda Patro	
		Dr. Brij Kishor Tiwari	
		Dr. J. Ravichander	
		Dr. Mohammand Ali Shaik	
		Dr. Anirodh Kumar	
		Mr. S. Naresh Kumar	
		Dr. RAJESH VELPULA	
		Mr. Kundhan Kumar	
		Ms. Ch. Rajitha	
		Mr. M Prakash	
		Mr. B. Raju	
		Intern 1 (Dharma teja)	
		Intern 2 (Sai Prasad)	
		Intern 3 (Sowmya)	
NS_2 (Mounika)			
CourseCode	24CS002PC215	CourseTitle	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	Week1 - Tuesday	Time(s)	
Duration	2 Hours	Applicable to Batches	24CSBTB01 To 24CSBTB39
AssignmentNumber: 1.2 (Present assignment number) / 24 (Total number of assignments)			
Q.No.	Question	Expected Time to complete	
1	Lab 1: Environment Setup – GitHub Copilot and VS Code Integration Lab Objectives: - To install and configure GitHub Copilot in Visual Studio Code. - To explore AI-assisted code generation using GitHub Copilot.	Week1 - wednesday	

- To analyze the accuracy and effectiveness of Copilot's code suggestions.
- To understand prompt-based programming using comments and code context

Lab Outcomes (LOs):

After completing this lab, students will be able to:

- Set up GitHub Copilot in VS Code successfully.
- Use inline comments and context to generate code with Copilot.
- Evaluate AI-generated code for correctness and readability.
- Compare code suggestions based on different prompts and programming styles.

Task Description#1

- Write a comment: # Function to check if a string is a valid palindrome (ignoring spaces and case) and allow Copilot to complete it.

Expected Output#1

- A function that correctly returns True for phrases like "A man a plan a canal Panama"

Prompt:

Function to check if a string is a valid palindrome (ignoring spaces and case)

The screenshot shows a Jupyter Notebook titled 'Colab ai assistant - Untitled0.ipynb - Colab'. The notebook contains a single code cell with the following Python code:

```
# Function to check if a string is a valid palindrome (ignoring spaces and case)
def is_palindrome(s):
    """
    Checks if a string is a valid palindrome, ignoring spaces and case.

    Args:
        s: The input string.

    Returns:
        True if the string is a palindrome, False otherwise.
    """
    # Remove spaces and convert to lowercase
    s = "".join(s.split()).lower()
    # Check if the string is equal to its reverse
    return s == s[::-1]

# Example usage
print(is_palindrome("A man a plan a canal Panama"))
print(is_palindrome("hello world"))
```

The output of the code cell is displayed below the code:

```
True
False
```

Observation:

the code cell, the `is_palindrome` function correctly identifies "A man a plan a canal Panama" as a palindrome (True) and "hello world" as not a palindrome (False).

Task Description#2

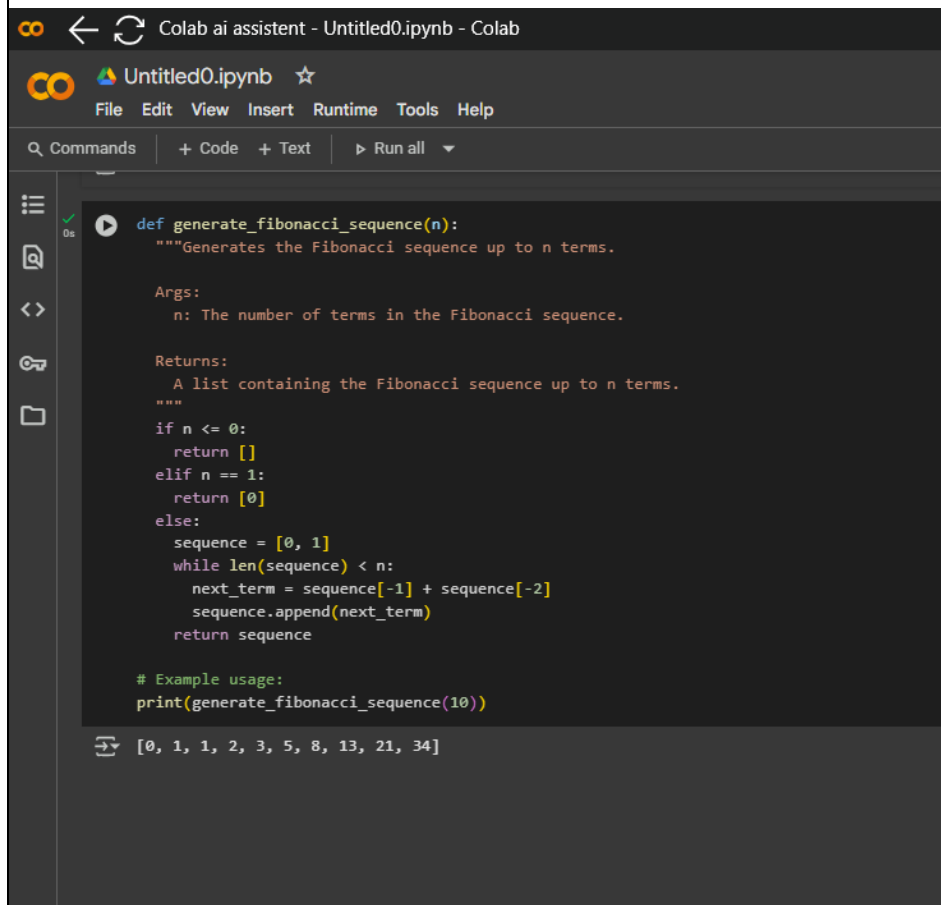
- Generate a Python function that returns the Fibonacci sequence up to n terms. Prompt with only a function header and docstring

Expected Output#2

- AI completes the function logic using loop or recursion with accurate output

Prompt:

function that returns the Fibonacci sequence up to n terms



The screenshot shows a Google Colab notebook interface. At the top, the title bar reads "Colab ai assistant - Untitled0.ipynb - Colab". Below it is a menu bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". A search bar contains "Commands" and there are buttons for "+ Code", "+ Text", and "Run all". The main code area has a dark background and contains the following Python code:

```
def generate_fibonacci_sequence(n):  
    """Generates the Fibonacci sequence up to n terms.  
  
    Args:  
        n: The number of terms in the Fibonacci sequence.  
  
    Returns:  
        A list containing the Fibonacci sequence up to n terms.  
    """  
    if n <= 0:  
        return []  
    elif n == 1:  
        return [0]  
    else:  
        sequence = [0, 1]  
        while len(sequence) < n:  
            next_term = sequence[-1] + sequence[-2]  
            sequence.append(next_term)  
        return sequence  
  
# Example usage:  
print(generate_fibonacci_sequence(10))
```

Below the code cell, the output is displayed: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34].

Obsetvation:

the code cell, the generate_fibonacci_sequence(10) function call successfully generated and printed the first 10 terms of the Fibonacci sequence: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34].

Task Description#3

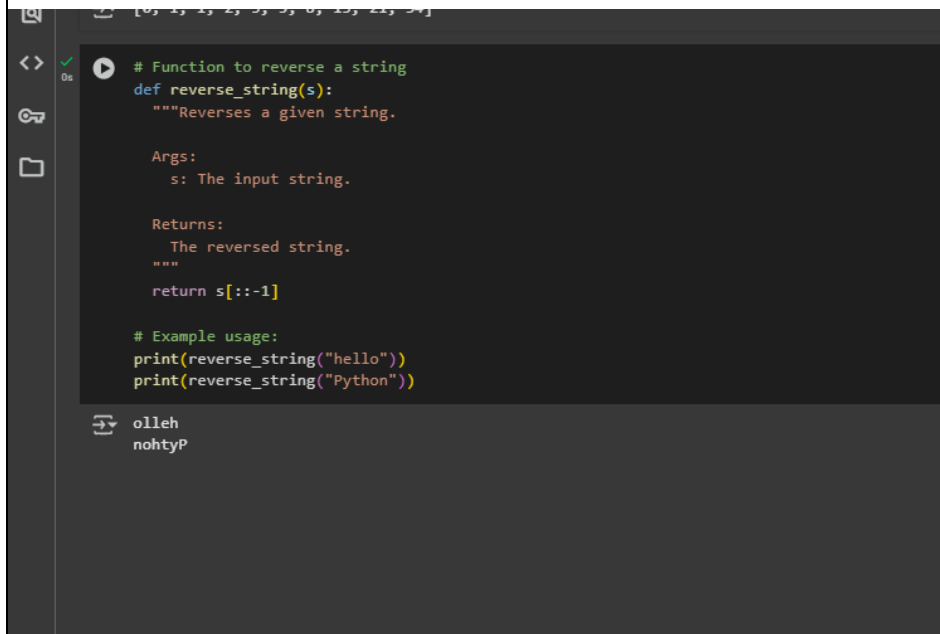
- Write a comment like # Function to reverse a string and use Copilot to generate the function.

Expected Output#3

- Auto-completed reverse function

Prompt:

Function to reverse a string



```
# Function to reverse a string
def reverse_string(s):
    """Reverses a given string.

    Args:
        s: The input string.

    Returns:
        The reversed string.
    """
    return s[::-1]

# Example usage:
print(reverse_string("hello"))
print(reverse_string("Python"))
```

olleh
nohtyP

Observation:

the code cell, the reverse_string function successfully reversed "hello" to "olleh" and "Python" to "nohtyP", as shown in the output.

Task Description#4

- Generate a program that simulates a basic calculator (add, subtract, multiply, divide). Write the comment: # Simple calculator with 4 operations and let AI complete it.

Expected Output#4

- Fully working calculator with input/output and operator selection logic

Prompt:

Simple calculator with 4 operations
a basic calculator (add, subtract, multiply, divide)

```
Colab ai assistant - Untitled0.ipynb - Colab

Untitled0.ipynb
File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all ▼

# Simple calculator with 4 operations

def add(x, y):
    return x + y

def subtract(x, y):
    return x - y

def multiply(x, y):
    return x * y

def divide(x, y):
    if y == 0:
        return "Error! Division by zero."
    return x / y

print("Select operation:")
print("1.Add")
print("2.Subtract")
print("3.Multiply")
print("4.Divide")

while True:
    choice = input("Enter choice(1/2/3/4): ")

    if choice in ('1', '2', '3', '4'):
        try:
            num1 = float(input("Enter first number: "))
            num2 = float(input("Enter second number: "))
        except ValueError:
            print("Invalid input. Please enter numbers.")
            continue

        if choice == '1':
            print(num1, "+", num2, "=", add(num1, num2))

        elif choice == '2':
            print(num1, "-", num2, "=", subtract(num1, num2))

        elif choice == '3':
            print(num1, "*", num2, "=", multiply(num1, num2))

        elif choice == '4':
            print(num1, "/", num2, "=", divide(num1, num2))

        next_calculation = input("Let's do next calculation? (yes/no): ")
        if next_calculation.lower() == "no":
            break

Colab ai assistant - Untitled0.ipynb - Colab

Untitled0.ipynb
File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all ▼

elif choice == '2':
    print(num1, "-", num2, "=", subtract(num1, num2))

elif choice == '3':
    print(num1, "*", num2, "=", multiply(num1, num2))

elif choice == '4':
    print(num1, "/", num2, "=", divide(num1, num2))

next_calculation = input("Let's do next calculation? (yes/no): ")
if next_calculation.lower() == "no":
    break
else:
    print("Invalid Input")

Select operation:
1.Add
2.Subtract
3.Multiply
4.Divide
Enter choice(1/2/3/4): 7
Invalid input
Enter choice(1/2/3/4): 3
Enter first number: 9
Enter second number: 5
9.0 * 5.0 = 45.0
Let's do next calculation? (yes/no): no
```

Observation:

the code cell, the calculator program successfully took input for an operation (3 for multiply) and two numbers (9 and 5). It then performed the multiplication and printed the result: $9.0 * 5.0 = 45.0$. The user then chose to stop the calculation.

Task Description#5

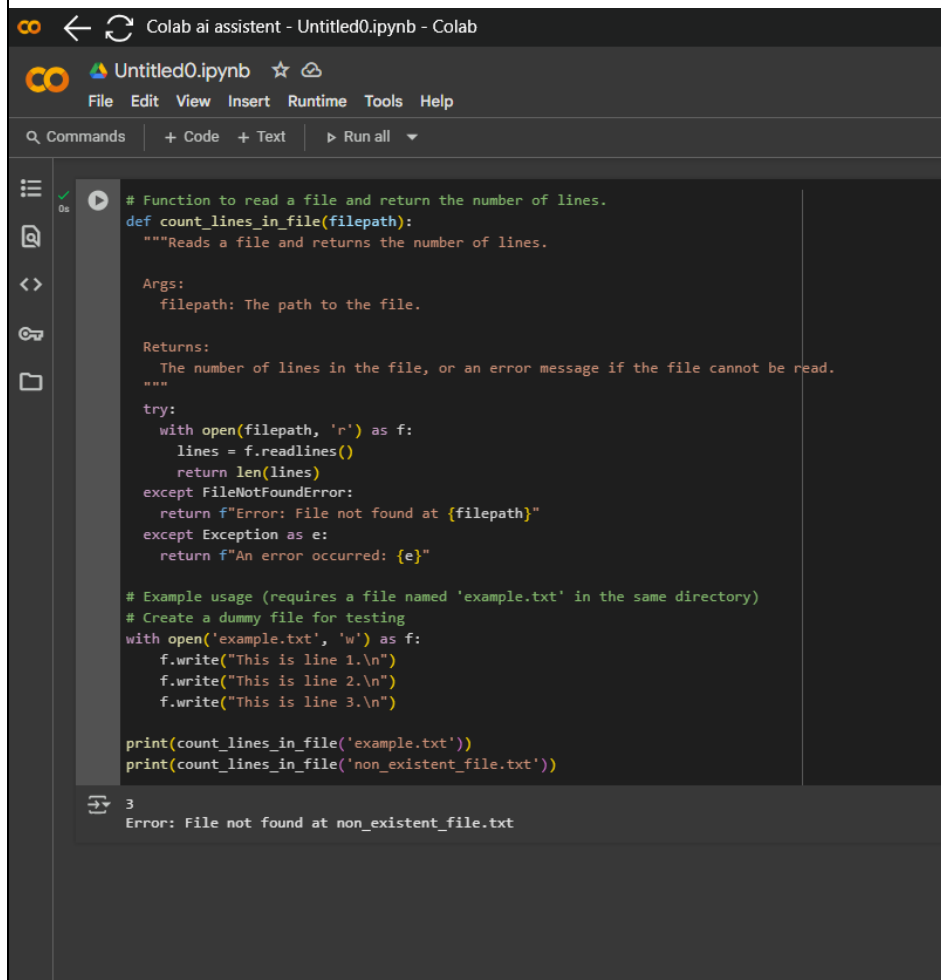
- Use a comment to instruct AI to write a function that reads a file and returns the number of lines..

Expected Output#5

- Functional implementation using open() or with open() and readlines()

Prompt:

write a function that reads a file and returns the number of lines..



The screenshot shows a Google Colab notebook interface. The title bar reads "Colab ai assistant - Untitled0.ipynb - Colab". Below the title bar is a menu bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". A toolbar shows "Commands", "+ Code", "+ Text", and "Run all". The main code cell contains the following Python code:

```
# Function to read a file and return the number of lines.
def count_lines_in_file(filepath):
    """Reads a file and returns the number of lines.

    Args:
        filepath: The path to the file.

    Returns:
        The number of lines in the file, or an error message if the file cannot be read.
    """
    try:
        with open(filepath, 'r') as f:
            lines = f.readlines()
            return len(lines)
    except FileNotFoundError:
        return f"Error: File not found at {filepath}"
    except Exception as e:
        return f"An error occurred: {e}"

# Example usage (requires a file named 'example.txt' in the same directory)
# Create a dummy file for testing
with open('example.txt', 'w') as f:
    f.write("This is line 1.\n")
    f.write("This is line 2.\n")
    f.write("This is line 3.\n")

print(count_lines_in_file('example.txt'))
print(count_lines_in_file('non_existent_file.txt'))
```

The output of the code cell is shown below the code:

```
3
Error: File not found at non_existent_file.txt
```

Observation:

the code cell, the `count_lines_in_file` function successfully counted 3 lines in 'example.txt'. When called with 'non_existent_file.txt', it correctly returned the error message "Error: File not found at non_existent_file.txt".

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots

Evaluation Criteria:

Criteria	Max Marks
Task #1	0.5
Task #2	0.5
Task #3	0.5
Task #4	0.5
Task #5	0.5
Total	2.5 Marks